

Data Structure Labs Report

Python Data Structure Labs | Assignment – 6907012660059 Theeradech Thaopet

Lab1

```
1 def main():
2     scores = []
3     for i in range(5):
4         while True:
5             try:
6                 score = int(input(f"Enter score {i + 1}: "))
7                 if 0 <= score <= 100:
8                     break
9                 print("Score must be between 0 and 100. Please try again.")
10            except ValueError:
11                print("Invalid input. Please enter a number.")
12            scores.append(score)
13
14
15     total = sum(scores)
16     average = total / len(scores)
17     highest = max(scores)
18     lowest = min(scores)
19     passed = sum(1 for score in scores if score >= 50)
20
21     print(f"All scores: {scores}")
22     print(f"Total: {total}")
23     print(f"Average: {average:.2f}")
24     print(f"Highest score: {highest}")
25     print(f"Lowest score: {lowest}")
26     print(f"Number of students who passed : {passed}")
27
28 if __name__ == "__main__":
29     main()
```

Short explanation:

This program stores scores for five users, validates the input format, and then calculates the Total Score, Average, Highest, Lowest, and number of users who passed.

The data structures:

First is list, holding integers, and this is only real data structure, because everything else (total, average, highest, lowest, passed) is a value complete form that list.

Screenshots:

```
Enter score 1: 45
Enter score 2: 50
Enter score 3: 150
Score must be between 0 and 100. Please try again.
Enter score 3: -1
Score must be between 0 and 100. Please try again.
Enter score 3: 59
Enter score 4: 30
Enter score 5: 89
All scores: [45, 50, 59, 30, 89]
Total: 273
Average: 54.60
Highest score: 89
Lowest score: 30
Number of students who passed : 3
> whitx@MacBook-Air-khxng-whitx Coding %
```

Problems encountered:

There were no problems at all when I did Lab 01; everything went smoothly. I always compile my work on GitHub, and if I encounter any problems, I fix and improve it.

Lab2

```
1 def main():
2     days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
3     temperatures = []
4     for day in days:
5         while True:
6             try:
7                 temp = float(input(f"Enter temperature for {day}: "))
8                 if -50 <= temp <= 60:
9                     break
10                print("Temperature must be between -50°C and 60°C. Please try again.")
11            except ValueError:
12                print("Invalid input. Please enter a number.")
13            temperatures.append((day, temp))
14
15    print("\nDaily Temperatures:")
16    for day, temp in temperatures:
17        print(f"{day}: {temp}°C")
18
19    total = sum(temp for day, temp in temperatures)
20    average = total / len(temperatures)
21    print(f"\nAverage temperature: {average:.2f}°C")
22
23    max_temp = max(temp for day, temp in temperatures)
24    min_temp = min(temp for day, temp in temperatures)
25
26    hottest_days = [day for day, temp in temperatures if temp == max_temp]
27    coolest_days = [day for day, temp in temperatures if temp == min_temp]
28
29    print(f"Hottest day: {' '.join(hottest_days)}, {max_temp}°C")
30    print(f"Coolest day: {' '.join(coolest_days)}, {min_temp}°C")
31
32    print("\nDays above average:")
33    for day, temp in temperatures:
34        if temp > average:
35            print(f"{day}: {temp}°C")
36
37    sorted_temperatures = sorted(temperatures, key=lambda x: x[1], reverse=True)
38    print("\nSorted by temperature (highest to lowest):")
39
40    i = 0
41    while i < len(sorted_temperatures):
42        current_temp = sorted_temperatures[i][1]
43        same_temp_days = [day for day, temp in sorted_temperatures if temp == current_temp]
44        print(f"{' '.join(same_temp_days)}: {current_temp}°C")
45        i += len(same_temp_days)
46
47    if __name__ == "__main__":
48        main()
```

Short explanation:

This program records paired temperature data (-50 to 60) across seven days, then displays the readings alongside the average, maximum, and minimum values. It identifies above-average days and presents temperatures in descending order, consolidating duplicates into a single line.

The data structures:

one list of tuples, each holding a day name paired with a temperature, is the only real data structure — everything else (total, average, max_temp, min_temp, hottest_days, sorted_temperatures) is derived from that one list.

Screenshots:

```
Enter temperature for Monday: 30
Enter temperature for Tuesday: 30
Enter temperature for Wednesday: 32
Enter temperature for Thursday: 90
Temperature must be between -50°C and 60°C. Please try again.
Enter temperature for Thursday: -60
Temperature must be between -50°C and 60°C. Please try again.
Enter temperature for Thursday: 35
Enter temperature for Friday: 34
Enter temperature for Saturday: 34
Enter temperature for Sunday: 31

Daily Temperatures:
Monday: 30.0°C
Tuesday: 30.0°C
Wednesday: 32.0°C
Thursday: 35.0°C
Friday: 34.0°C
Saturday: 34.0°C
Sunday: 31.0°C

Average temperature: 32.29°C
Hottest day: Thursday, 35.0°C
Coolest day: Monday, Tuesday, 30.0°C

Days above average:
Thursday: 35.0°C
Friday: 34.0°C
Saturday: 34.0°C

Sorted by temperature (highest to lowest):
Thursday: 35.0°C
Friday, Saturday: 34.0°C
Wednesday: 32.0°C
Sunday: 31.0°C
Monday, Tuesday: 30.0°C
whitx@MacBook-Air-khxng-whitx Coding %
```

Problems encountered:

The first problem I encountered was that if the temperature was the same each day, it would always appear on a different line. I've fixed that now, and I haven't encountered any other problems since.

Lab3

```
1  def main():
2      student = {}
3      student["student_id"] = input("Enter student ID: ")
4      student["name"] = input("Enter name: ")
5      student["major"] = input("Enter major: ")
6
7      scores = []
8      for i in range(3):
9          while True:
10             try:
11                 score = int(input(f"Enter score {i + 1}: "))
12                 if 0 <= score <= 100:
13                     break
14                 print("Score must be between 0 and 100. Please try again.")
15             except ValueError:
16                 print("Invalid input. Please enter a number.")
17             scores.append(score)
18
19      student["scores"] = scores
20      total = sum(scores)
21      average = total / len(scores)
22      student["average"] = average
23      if average >= 80:
24          student["grade"] = "A"
25      elif average >= 75:
26          student["grade"] = "B+"
27      elif average >= 70:
28          student["grade"] = "B"
29      elif average >= 65:
30          student["grade"] = "C+"
31      elif average >= 60:
32          student["grade"] = "C"
33      elif average >= 55:
34          student["grade"] = "D+"
35      elif average >= 50:
36          student["grade"] = "D"
37      else:
38          student["grade"] = "F"
39
40      print(f"Student ID: {student['student_id']}")
41      print(f"Name: {student['name']}")
42      print(f"Major: {student['major']}")
43      print(f"Scores: {scores[0]}, {scores[1]}, {scores[2]}")
44      print(f"Average: {student['average']:.2f}")
45      print(f"Grade: {student['grade']}")
46
47  if __name__ == "__main__":
48      main()
```

Short explanation:

This program stores a student's ID, name, major, and three subject scores in a dictionary. It collects the scores, computes the grade, evaluates it against defined criteria, and displays the retrieved results.

The data structures:

one dictionary, holding a nested list of scores plus scalar fields (id, name, major, average, grade), is the only real data structure — everything else is just filled into that one dictionary.

Screenshots:

```
Enter student ID: 6907012660059
Enter name: Theeradech Thaopet
Enter major: I-BIT
Enter score 1: 45
Enter score 2: 120
Score must be between 0 and 100. Please try again.
Enter score 2: -129
Score must be between 0 and 100. Please try again.
Enter score 2: 70
Enter score 3: 79
Student ID: 6907012660059
Name: : Theeradech Thaopet
Major: I-BIT
Scores: 45, 70, 79
Average: 64.67
Grade: C
```

Problems encountered:

No problems were encountered in this lab; everything went well.

Lab4

```
1 def main():
2     inventory = {}
3
4     while True:
5         print("1. Add a product")
6         print("2. Update product quantity")
7         print("3. Remove a product")
8         print("4. Search for a product")
9         print("5. Show all products")
10        print("6. Show low-stock products")
11        print("7. Calculate inventory value")
12        print("8. Exit")
13        choice = input("Enter your choice: ")
14
15        if choice == "1":
16            code = input("Enter product code: ")
17            if code in inventory:
18                print("This product code already exists.")
19            else:
20                name = input("Enter product name: ")
21                while True:
22                    try:
23                        price = float(input("Enter price: "))
24                        if price >= 0:
25                            break
26                        print("Price must be >= 0. Please try again.")
27                    except ValueError:
28                        print("Invalid input. Please enter a number.")
29                while True:
30                    try:
31                        quantity = int(input("Enter quantity: "))
32                        if quantity >= 0:
33                            break
34                        print("Quantity must be >= 0. Please try again.")
35                    except ValueError:
36                        print("Invalid input. Please enter a number.")
37                inventory[code] = {"name": name, "price": price, "quantity": quantity}
38                print("Product added.")
39
40        elif choice == "2":
41            code = input("Enter product code: ")
42            if code in inventory:
43                print("a. Stock in (add to quantity)")
44                print("b. Stock out (remove from quantity)")
45                sub_choice = input("Choose transaction type (a/b): ")
46
47                if sub_choice == "a":
48                    while True:
49                        try:
50                            amount = int(input("Enter quantity to stock in: "))
51                            break
52                        except ValueError:
53                            print("Invalid input. Please enter a number.")
54                        if amount < 0:
55                            print("Stock-in amount cannot be negative.")
56                        else:
57                            inventory[code]["quantity"] += amount
58                            print(f"Stocked in (amount). New quantity: {inventory[code]['quantity']}")
59
60                elif sub_choice == "b":
61                    while True:
62                        try:
63                            amount = int(input("Enter quantity to stock out: "))
64                            break
65                        except ValueError:
66                            print("Invalid input. Please enter a number.")
67                        if amount < 0:
68                            print("Stock-out amount cannot be negative.")
69                        elif amount > inventory[code]["quantity"]:
70                            print(f"Not enough stock. Current quantity: {inventory[code]['quantity']}")
71                        else:
72                            inventory[code]["quantity"] -= amount
73                            print(f"Stocked out (amount). New quantity: {inventory[code]['quantity']}")
74
75                else:
76                    print("Invalid transaction type.")
77            else:
78                print("Product not found.")
79
80        elif choice == "3":
81            code = input("Enter product code: ")
82            if code in inventory:
83                del inventory[code]
84                print("Product removed.")
85            else:
86                print("Product not found.")
87
88        elif choice == "4":
89            code = input("Enter product code: ")
90            if code in inventory:
91                product = inventory[code]
92                print(f"Name: {product['name']}")
93                print(f"Price: {product['price']}")
94                print(f"Quantity: {product['quantity']}")
95            else:
96                print("Product not found.")
97
98        elif choice == "5":
99            if inventory:
100                print("\n--- All Products ---")
101                for code, product in inventory.items():
102                    print(f"{code} - {product['name']} | Price: {product['price']} | Quantity: {product['quantity']}")
103            else:
104                print("No products in inventory.")
105
106        elif choice == "6":
107            print("\n--- Low-Stock Products ---")
108            found = False
109            for code, product in inventory.items():
110                if product["quantity"] < 5:
111                    print(f"{code} - {product['name']} | Quantity: {product['quantity']}")
112                    found = True
113            if not found:
114                print("No low-stock products.")
115
116        elif choice == "7":
117            print("\n--- Inventory Value ---")
118            total = 0
119            for code, product in inventory.items():
120                value = product["price"] * product["quantity"]
121                print(f"{code} - {product['name']} = {value}")
122                total += value
123            print(f"Total inventory value: {total}")
124
125        elif choice == "8":
126            print("Goodbye!")
127            break
128        else:
129            print("Invalid choice.")
130
131 if __name__ == "__main__":
132     main()
133
```

Short explanation:

This program manages inventory data using a dictionary structure, with product codes as keys and corresponding name, price, and quantity values, operating within a menu-driven loop until termination (Option 8). Functionalities include adding products with duplicate validation (Option 1), modifying stock quantities with input verification (Option 2), removing products (Option 3), displaying individual product details (Option 4), listing all inventory items (Option 5), searching for specific products (Option 6), and computing total inventory value (Option 7).

The data structures:

one nested dictionary is the only real data structure. Inventory maps a product code to another dictionary containing name, price, and quantity. Everything else, like total, found, and value, is just computed by looping through it.

Screenshots:

```
1. Add a product
2. Update product quantity
3. Remove a product
4. Search for a product
5. Show all products
6. Show low-stock products
7. Calculate inventory value
8. Exit
Enter your choice: 1
Enter product code: 001
Enter product name: Doritos
Enter price: 75
Enter quantity: 75
Product added.
Enter your choice: 5

--- All Products ---
001 - Doritos | Price: 75.0 | Quantity: 75
002 - Lay | Price: 35.0 | Quantity: 50
003 - Testo | Price: 25.0 | Quantity: 100
Enter your choice: 2
Enter product code: 003
  a. Stock in (add to quantity)
  b. Stock out (remove from quantity)
  Choose transaction type (a/b): a
Enter quantity to stock in: 25
Stocked in 25. New quantity: 125
Enter your choice: 2
Enter product code: 003
  a. Stock in (add to quantity)
  b. Stock out (remove from quantity)
  Choose transaction type (a/b): b
Enter quantity to stock out: 124
Stocked out 124. New quantity: 1
Enter your choice: 4
Enter product code: 003
Name: Testo
Price: 25.0
Quantity: 1
```



```
Enter your choice: 6
--- Low-Stock Products ---
003 - Testo | Quantity: 1
--- Inventory Value ---
001 - Doritos = 5625.0
002 - Lay = 1750.0
003 - Testo = 25.0
Total inventory value: 7400.0
Enter your choice: 3
Enter product code: 003
Product removed.
Enter your choice: 5
--- All Products ---
001 - Doritos | Price: 75.0 | Quantity: 75
002 - Lay | Price: 35.0 | Quantity: 50
Enter your choice: 8
Goodbye!
```

Problems encountered:

I forgot to add input data validation and error handling in some of the code, which allowed users to input unwanted data.